

TOBB Ekonomi ve Teknoloji Üniversitesi

Bilgisayar Mühendisliği Bölümü

Proje Teklifi (BİL 495)

FreeRTOS Tabanlı Denge Robotunda Kenar Yapay Zekâ Çıkarımının Gerçek Zamanlı Kontrol Döngüsü Üzerindeki Zamanlama Etkisinin Deneysel Analizi

Danışman Sadık Eğri

	Ali Hakan Kincal	–	221101064
Takım Üyeleri	Uğur Talan	–	211101065
	Mert Can Ayhan	–	221101055
	Barış Emre Ahi	–	221101045

Tarih: 21 Haziran 2026

Özet

Bu projede, ARM Cortex-M tabanlı bir mikrodenetleyici üzerinde FreeRTOS gerçek zamanlı işletim sistemi çalıştıran, kendi dengesini kurabilen iki tekerlekli bir robot geliştirilecektir. Robot, bir IMU sensöründen aldığı eğim verisini kullanarak bir PID kontrol döngüsüyle dik konumunu koruyacaktır. Projenin temel amacı yalnızca çalışan bir denge robotu üretmek değil; kaynak-kısıtlı bir gömülü sistemde *karma kritiklikteki* (mixed-criticality) iki iş yükünün – yüksek öncelikli gerçek zamanlı kontrol görevi ile TinyML tabanlı kenar yapay zekâ çıkarım görevi (zemin/yüzey sınıflandırması) – aynı işlemci çekirdeğini paylaştığı durumda, yapay zekâ iş yükünün kontrol döngüsünün zamanlama determinizmini ne ölçüde ve *hangi mekanizmalarla* bozduğunu deneysel olarak ölçmektir.

Bu kapsamda kontrol döngüsündeki jitter, yürütme süresi, deadline karşılama oranı, yapay zekâ çıkarım süresi ve robotun denge kalitesi (eğim açısı RMS hatası) ölçülecektir. Yapay zekâ görevi farklı öncelik seviyelerinde, farklı zamanlama politikalarıyla ve farklı iş yükü seviyelerinde çalıştırılarak senaryolar karşılaştırılacaktır. Tek çekirdekli bir sistemde girişimin sabit-öncelikli zamanlayıcı altında nereden kaynaklandığı (cache kirlenmesi, veri yolu çekişmesi, kritik bölümler ve kesme gecikmesi) açıkça hipotezleştirilecek; bu nedenle deneyler önbellekli (I/D-cache içeren) bir Cortex-M sınıfı üzerinde yürütülecektir. Ayrıca mümkün olduğu durumda MPU tabanlı bellek izolasyonu kullanılarak yapay zekâ görevinin kritik kontrol görevlerinden ayrıştırılması incelenecektir. Ölçümlerde istatistiksel titizlik (kuyruk gecikmeleri, dağılım analizi, tekrarlanabilirlik) ve karıştırıcı değişken kontrolü esas alınacaktır. Proje sonunda çalışan bir robot prototipi, deneysel zamanlama verileri, karşılaştırmalı analizler, teknik rapor, açık kaynak kod deposu ve demo sunumu elde edilmesi hedeflenmektedir.

Anahtar Kelimeler: FreeRTOS, gerçek zamanlı işletim sistemi, gömülü/kenar yapay zeka, PID kontrol, zamanlama jitter'ı, deadline, MPU bellek izolasyonu, zemin sınıflandırması.

İçindekiler

Özet	1
1 Problem Tanımı ve Motivasyon	3
2 Hedef Kullanıcı ve Etki Alanı	3
3 Önceki Çalışmalar ve Literatür	4
3.1 Kenar Yapay Zekâ ve TinyML	4
3.2 İki Tekerlekli Denge Robotları	4
3.3 Gerçek Zamanlı İşletim Sistemleri ve FreeRTOS	4
3.4 Zamanlama Girişimi, Determinizm ve İzolasyon	5
3.5 Literatürdeki Boşluk ve Bu Projenin Katkısı	5
4 Çözüm Yaklaşımı ve Yöntem	6
4.1 Kontrol Yaklaşımı	6
4.2 Gerçek Zamanlı Görev Yapısı	6
4.3 TinyML Yapay Zekâ Görevi: Zemin Sınıflandırması	7
4.4 Girişim Mekanizmaları Hipotezi	8
4.5 Ölçüm Yöntemi	8
5 Mimari Taslak	8
6 Deney Senaryoları	9
7 Kapsam ve Başarı Kriterleri	9
7.1 Kapsam Dahili Konular	9
7.2 Kapsam Dışı Konular	10
7.3 Ölçülebilir Başarı Kriterleri	10
8 Çalışma Planı ve İş Bölümü	10
8.1 Çalışma Planı	10
8.2 İş Bölümü	11
9 Riskler ve Önlemler	12
10 Beklenen Çıktılar	13
11 Sonuç	14
Kaynaklar	14

1 Problem Tanımı ve Motivasyon

Günümüzde yapay zekâ uygulamaları yalnızca bulut sistemlerinde değil, doğrudan gömülü ve kenar cihazlar üzerinde de çalıştırılmaktadır. Bu yaklaşım; düşük gecikme, ağ bağlantısına daha az bağımlılık, veri gizliliği ve yerel karar alma gibi avantajlar sağlamaktadır [1, 6]. Mikrodenetleyici ve düşük güçlü kenar platformlarında makine öğrenmesi çıkarımının yaygınlaşması, hem akademik hem endüstriyel literatürde hızla büyüyen bir alan haline gelmiştir [3, 4]. Ancak mikrodenetleyici tabanlı sistemler işlemci zamanı, bellek kapasitesi ve enerji tüketimi açısından kısıtlı kaynaklara sahiptir. Bu nedenle yapay zekâ çıkarım görevlerinin gerçek zamanlı kontrol görevleriyle *aynı işlemci üzerinde* çalıştırılması önemli bir mühendislik problemi oluşturmaktadır.

Gerçek zamanlı kontrol sistemlerinde yalnızca hesaplamanın doğru yapılması yeterli değildir; hesaplamanın belirli bir süre içerisinde tamamlanması da gerekir [21]. Kontrol döngüsünün gecikmesi, periyodunun bozulması veya deadline değerini kaçırmaması sistem kararlılığını doğrudan etkileyebilir; geri beslemeli kontrolde zamanlama gecikmeleri kontrol performansını ve kararlılık marjlarını düşürür [20]. Bu durum özellikle robotik, otomotiv, savunma, medikal cihazlar ve endüstriyel otomasyon gibi emniyet-kritik alanlarda önemlidir. Siber-fiziksel sistemlerde kesme gecikmesi ve giriş/çıkış olaylarının öngörülebilirliği başlı başına ölçülmesi gereken bir nicelik haline gelmiştir [7].

Bu projede iki tekerlekli bir denge robotu deney platformu olarak seçilmiştir. Bunun nedeni, kontrol döngüsündeki zamanlama bozulmalarının *fiziksel olarak gözlemlenebilir* olmasıdır. Eğer kontrol görevi zamanında çalışmazsa robotun eğim açısı hızla değişebilir ve robot dengesini kaybedebilir. Böylece jitter, deadline kaçırma ve görev önceliği gibi soyut gerçek zamanlı sistem kavramları fiziksel bir sistem üzerinde somut olarak ve ölçülebilir bir çıktı (eğim açısı RMS hatası) ile değerlendirilebilir.

Projenin çözmek istediği temel problem şudur:

Kaynak-kısıtlı bir ARM Cortex-M mikrodenetleyici üzerinde çalışan gerçek zamanlı kontrol sistemi ile TinyML tabanlı yapay zekâ çıkarım görevi bir arada çalıştırıldığında, yapay zekâ iş yükü kontrol döngüsünün zamanlama determinizmini ne ölçüde ve hangi mekanizmalarla bozar; bu etki görev önceliklendirmesi, zamanlama stratejileri ve bellek izolasyonu ile nasıl sınırlandırılabilir?

Bu problem günceldir, çünkü yapay zekâ destekli gömülü sistemlerin sayısı artmakta; ancak bu sistemlerde yapay zekâ görevlerinin gerçek zamanlı kontrol görevleriyle güvenli ve öngörülebilir biçimde nasıl birlikte çalıştırılacağı hâlen açık bir araştırma ve mühendislik konusudur. Çok çekirdekli platformlarda paylaşılan kaynak çekişmesinin gerçek zamanlı davranışı bozduğu literatürde gösterilmiştir [2]; bu projede benzer girişim olgusunun *tek çekirdekli*, kaynak-kısıtlı bir mikrodenetleyicide nasıl ortaya çıktığı küçük ölçekli ve fiziksel bir sistem üzerinde incelenecektir.

2 Hedef Kullanıcı ve Etki Alanı

Bu projenin birincil hedef kullanıcıları; gömülü sistemler, gerçek zamanlı işletim sistemleri, robotik kontrol ve kenar yapay zekâ alanlarında çalışan öğrenciler, araştırmacılar ve mühendislerdir. Proje çıktıları, bu alanlarda deneysel çalışma yapmak isteyen kişiler için örnek bir platform ve yeniden kullanılabilir bir

ölçüm altyapısı sunacaktır.

İkincil hedef kullanıcılar ise emniyet-kritik sistemlerde yapay zekâ entegrasyonu üzerinde çalışan savunma, otomotiv, endüstriyel otomasyon ve robotik ekipleridir. Proje ticari bir ürün geliştirmeyi hedeflemese de, gerçek zamanlı kontrol ve yapay zekâ iş yüklerinin birlikte çalıştırılmasında karşılaşılabilecek zamanlama problemlerini küçük ölçekli bir fiziksel sistem üzerinde göstermeyi amaçlamaktadır.

Projenin etki alanları şunlardır:

- Gerçek zamanlı gömülü sistemler ve karma kritiklik (mixed-criticality)
- Kenar yapay zekâ ve TinyML
- Robotik kontrol sistemleri
- FreeRTOS tabanlı görev zamanlama
- Emniyet-kritik sistemlerde görev ve bellek izolasyonu
- Deneysel performans ölçümü ve sistem analizi

3 Önceki Çalışmalar ve Literatür

Bu proje, dört farklı alanın kesişiminde konumlanmaktadır: iki tekerlekli denge robotları, gerçek zamanlı işletim sistemleri, TinyML çıkarımı ve gerçek zamanlı zamanlama girişi / görev izolasyonu.

3.1 Kenar Yapay Zekâ ve TinyML

Kenar yapay zekâ, hesaplamayı veri kaynağına yaklaştırarak gecikmeyi azaltan ve gizliliği artıran bir paradigmadır [1, 6]. Derin sinir ağı çıkarımının kenar cihazlarda hızlandırılması ve buluta göre talebe bağlı dağıtımı üzerine kapsamlı çalışmalar bulunmaktadır [3, 8]. Cihaz üstü (on-device) makine öğrenmesi, gerçek zamanlı işleme gereksinimleri açısından ayrıca ele alınmaktadır [4]. Mikrodenetleyici ölçeğinde ise TinyML, model boyutu, bellek kullanımı, güç tüketimi ve çıkarım gecikmesi gibi metriklere odaklanır [17]. TensorFlow Lite Micro, mikrodenetleyiciler ve DSP benzeri kaynak-kısıtlı cihazlarda çıkarım yapabilmek için yaygın kullanılan bir altyapıdır [5]; bu tür sistemlerin standart karşılaştırması için MLPerf Tiny gibi ölçütler önerilmiştir [18]. Ancak bu çalışmaların çoğu çıkarım performansına odaklanır; çıkarımın *aynı çekirdekte koşan kritik bir kontrol görevine* zamanlama etkisini nadiren ele alır.

3.2 İki Tekerlekli Denge Robotları

İki tekerlekli denge robotları genellikle ters sarkaç modeli üzerinden ele alınır. Bu sistemlerde IMU sensörleri kullanılarak robotun eğim açısı tahmin edilir ve PID gibi geri beslemeli kontrol yöntemleriyle motorlar sürülür. Literatürde complementary filter, Kalman filter ve benzeri filtreleme yöntemlerinin IMU tabanlı açı kestirimi için kullanıldığı çalışmalar bulunmaktadır [19]. Bu çalışmalar genellikle robotun dengede tutulmasına odaklanır; gerçek zamanlı işletim sistemi zamanlamasının veya ek bir yapay zekâ iş yükünün denge üzerindeki etkisini incelemeyiz.

3.3 Gerçek Zamanlı İşletim Sistemleri ve FreeRTOS

FreeRTOS, mikrodenetleyici tabanlı sistemlerde görev yönetimi, sabit-öncelikli kesmeli (preemptive) zamanlama, kuyruklar, semaforlar ve bellek koruma birimi (MPU) desteği sunan yaygın bir RTOS

çözümüdür [16]. FreeRTOS'un görevler arası iletişim mekanizmasının doğruluğu formal yöntemlerle de incelenmiştir [11]. IoT işletim sistemlerinde kaynak yönetimi geniş bir literatüre sahiptir [15]; Zephyr gibi alternatif RTOS'ların gerçek zamanlı veri yönetimi için kullanımı da gösterilmiştir [12]. FreeRTOS üzerinde görev önceliklerinin doğru belirlenmesi, kritik kontrol görevlerinin zamanında çalışması açısından belirleyicidir.

3.4 Zamanlama Girişimi, Determinizm ve İzolasyon

Gerçek zamanlı sistemlerde kritik görevlerin belirlenen süre sınırları içinde çalışması temel bir güvenilirlik gereksinimidir [21]. Çok çekirdekli ve MPSoC tabanlı platformlarda paylaşılan bellek ve veri yolu çekişmesinin gerçek zamanlı davranışı bozduğu; bu nedenle akış (streaming) segmentlerinin hızlandırılması ve izolasyonu için özel yöntemler gerektiği gösterilmiştir [2]. Siber-fiziksel sistemlerde kesme gecikmesi ve olası verim, ölçülebilir nicelikler olarak ele alınmıştır [7]. Determinizmi yeniden üretmek (deterministic replay) çok çekirdekli RTOS uygulamalarında hata ayıklama için önerilmiştir [13]. Ayrıca güç yönetimi ve voltaj/frekans ölçekleme ile gerçek zamanlı zamanlama arasındaki etkileşim, determinizmi etkileyen bir faktör olarak incelenmiştir [10]. Bu çalışmalar girişimin ve determinizm kaybının önemini ortaya koysa da, çoğunlukla büyük ölçekli veya çok çekirdekli platformlara odaklanır; küçük, tek çekirdekli bir mikrodenetleyicide bir AI iş yükünün fiziksel bir kontrol döngüsüne etkisini somut ve ölçülebilir biçimde göstermez.

3.5 Literatürdeki Boşluk ve Bu Projenin Katkısı

Mevcut çalışmaların önemli bir kısmı ya yalnızca denge kontrolüne, ya yalnızca TinyML çıkarım performansına, ya da yalnızca RTOS görev yönetimine odaklanmaktadır. Bu projenin farkı, gerçek zamanlı kontrol ve TinyML çıkarım görevlerini *aynı fiziksel robot platformu ve aynı çekirdek* üzerinde bir araya getirerek, yapay zekâ iş yükünün kontrol döngüsü üzerindeki zamanlama etkisini – ve bu etkinin mekanizmalarını – deneysel olarak ölçmesidir. Tablo 1 önceki çalışmalarla bu projeyi karşılaştırmaktadır.

Tablo 1: Önceki çalışmalar ile önerilen projenin karşılaştırılması

Çalışma / Alan	Odak Noktası	Sınırlılık	Bu Projenin Farkı
Denge robotu çalışmaları	IMU, PID, motor kontrolü	AI yükü ve RTOS zamanlama etkisi genellikle incelenmez	Denge robotu gerçek zamanlı zamanlama deney platformu olarak kullanılır
FreeRTOS uygulamaları	Görev yönetimi, önceliklendirme, RTOS mekanizmaları	Fiziksel kontrol kalitesine etkisi her zaman ölçülmez	Görev zamanlamasının robot dengesi üzerindeki etkisi ölçülür
TinyML çalışmaları	MCU'da AI çıkarımı, model boyutu, gecikme	Gerçek zamanlı kontrol göreviyle etkileşim sınırlı incelenir	AI çıkarımının kontrol jitter ve deadline üzerindeki etkisi ölçülür
Girişim / izolasyon yaklaşımları	Paylaşılan kaynak çekişmesi, bellek koruma, izolasyon	Çoğunlukla çok çekirdekli; fiziksel kontrol sistemiyle gösterilmez	Tek çekirdekli MCU'da AI görevinin kritik kontrolden ayrıştırılması deneysel ele alınır

4 Çözüm Yaklaşımı ve Yöntem

Projenin çözüm yaklaşımı, fiziksel bir denge robotu üzerinde deneysel ölçüm yapmaya dayanmaktadır. Sistem önce yapay zekâ görevi olmadan kararlı çalışacak şekilde tasarlanacak, ardından TinyML çıkarım görevi sisteme eklenerek farklı zamanlama senaryoları karşılaştırılacaktır. Genel yöntem aşağıdaki adımlardan oluşmaktadır:

1. Önbellekli bir ARM Cortex-M tabanlı mikrodenetleyici üzerinde temel çevre birimleri ve FreeRTOS altyapısı kurulacaktır.
2. IMU sensöründen eğim verisi alınacak ve filtreleme yöntemiyle robotun açı tahmini yapılacaktır.
3. PID kontrol algoritması geliştirilerek motorlar PWM sinyalleriyle sürülecektir.
4. Robot, AI görevi olmadan temel denge kontrolü sağlayacak hâle getirilecektir.
5. Kontrol döngüsünün jitter, yürütme süresi ve deadline karşılama oranı ölçülecektir (referans S0 senaryosu).
6. Zemin sınıflandırması yapan TinyML çıkarım görevi sisteme ayrı bir FreeRTOS görevi olarak eklenecektir.
7. AI görevi farklı öncelik, zamanlama politikası ve iş yükü seviyeleriyle çalıştırılacaktır.
8. Her senaryoda kontrol döngüsü ve robot denge kalitesi ölçülerek karşılaştırılacaktır.
9. Mümkün olduğu durumda MPU tabanlı bellek izolasyonu ile AI görevinin kritik kontrol verilerine erişimi sınırlandırılacaktır.

4.1 Kontrol Yaklaşımı

Robotun denge kontrolü için IMU sensöründen elde edilen ivmeölçer ve jiroskop verileri kullanılacaktır. Bu verilerden robotun eğim açısı hesaplanacaktır. İlk aşamada complementary filter gibi düşük maliyetli bir filtreleme yaklaşımı kullanılacak; gerekli görülürse Kalman filter benzeri alternatifler değerlendirilecektir [19]. Kontrol algoritması olarak PID kullanılacaktır. PID, robotun hedef dik konumu ile ölçülen açısı arasındaki hatayı kullanarak motorlara uygulanacak PWM değerini belirleyecektir. Kontrol döngüsü mümkün olduğunca sabit periyotla çalıştırılacak ve her döngünün yürütme süresi ölçülecektir. Deneylerin geçerliliği açısından PID kazançları tüm senaryolarda *sabit* tutulacak; böylece denge kalitesindeki değişimin kontrolcü ayarından değil, zamanlama girişiminden kaynaklandığı izole edilebilecektir.

4.2 Gerçek Zamanlı Görev Yapısı

Sistem FreeRTOS üzerinde birden fazla görevden oluşacaktır. Kritik kontrol görevi yüksek öncelikli ve periyodik olarak çalıştırılacaktır. AI görevi ise deney senaryosuna bağlı olarak farklı önceliklerde çalıştırılacaktır. Önerilen görev yapısı Tablo 2’de verilmiştir.

Tablo 2: Önerilen FreeRTOS görev yapısı

Görev	Öncelik	Sorumluluk
Control Task	En yüksek	IMU okuma, açı hesaplama, PID kontrol, motor PWM güncelleme
AI Task	Senaryoya bağlı	Zemin sınıflandırması (TinyML çıkarımı), üç yük seviyesinde
Telemetry Task	Düşük	Ölçüm verilerini UART/USB üzerinden PC'ye aktarma
Monitor Task	Orta	Deadline miss, sistem sağlığı ve güvenli durdurma kontrolü
Logging Task	En düşük	Ölçüm verilerini tamponlama ve kaydetme

4.3 TinyML Yapay Zekâ Görevi: Zemin Sınıflandırması

Bu projede TinyML görevi olarak *zemin (yüzey) sınıflandırması* belirlenmiştir. Robot hareket hâlindeyken IMU'nun ivmeölçer ve jiroskop verilerinde, üzerinde ilerlenen yüzeyin türüne (örneğin halı, parke, fayans, beton) bağlı olarak farklı titreşim imzaları oluşur. AI görevi, sabit uzunlukta bir IMU penceresini (örneğin 1–2 saniyelik örnek dizisi) girdi alarak yüzeyin sınıfını tahmin eder.

Bu görev iki nedenle seçilmiştir. Birincisi, *anlamlı bir iştir*: salt yapay bir işlem yükü üretmek yerine robotun çevresine ilişkin gerçek bir çıkarım yapar ve projenin denge temasıyla doğal biçimde ilişkilendirir. İkincisi, *tekrarlanabilir ve ölçeklenebilir bir CPU yüküdür*: çıkarım, model boyutu büyütülerek hafiften ağıra ölçeklenebildiği için, AI iş yükünün kontrol döngüsü üzerindeki zamanlama etkisini kademeli olarak incelemeye uygundur.

Önemli bir tasarım kararı olarak, zemin sınıflandırmasının çıktısı *kontrol döngüsüne geri beslenmez*; yalnızca telemetri olarak kaydedilir ve PC tarafında raporlanır. Böylece AI görevi kritik yolun dışında tutulur ve deney, “AI iş yükünün kontrol döngüsüne yalnızca zamanlama girişimi üzerinden etkisi” sorusuna sadık kalır. Zemin tahminine göre PID kazançlarının uyarlanması gibi kapalı-çevrim bir kullanım, ileride değerlendirilebilecek kapsam dışı bir genişletme olarak görülmektedir.

Çıkarımın tekrarlanabilirliğini garanti altına almak için, deney koşulları sırasında önceden kaydedilmiş sabit bir IMU penceresi kümesi modele verilecektir; böylece çıkarım süresi senaryolar arasında karşılaştırılabilir kalır ve girdi değişkenliği bir karıştırıcı (confounder) olmaktan çıkar. Zemin verisi çevrimdışı toplanacak, etiketlenecek ve model PC ortamında eğitilip mikrodenetleyiciye gömülecektir.

Projenin ana hedefi yüksek doğruluklu bir sınıflandırıcı geliştirmek değil, AI çıkarımının gerçek zamanlı kontrol üzerindeki zamanlama etkisini ölçmektir; zemin sınıflandırma doğruluğu ikincil bir metrik olarak raporlanacaktır. Bu nedenle aynı zemin sınıflandırma görevi, üç farklı model boyutuyla üç yük seviyesinde ele alınacaktır:

- **Hafif yük:** küçük bir MLP (birkaç bin MAC işlemi)
- **Orta yük:** IMU penceresi üzerinde 1B evrimsel ağ (1D-CNN)
- **Ağır yük:** daha derin bir 1D-CNN veya küçük bir zamansal model

Çıkarım için TensorFlow Lite Micro [5] temel alınacak; Cortex-M üzerinde daha öngörülebilir ve kısa çıkarım süresi için CMSIS-NN çekirdekleri ve int8 nicemleme (quantization) değerlendirilecektir. Bu sayede model karmaşıklığı arttıkça kontrol döngüsünün jitter ve deadline davranışının nasıl değiştiği gözlemlenebilecektir.

4.4 Girişim Mekanizmaları Hipotezi

Tek çekirdekli, sabit-öncelikli ve kesmeli bir zamanlayıcıda, en yüksek öncelikli kontrol görevi hazır olduğunda düşük öncelikli AI görevini doğrudan keser. Dolayısıyla girişim, yalnızca yanlış önceliklendirilmeden değil, öncelikten bağımsız *paylaşılan kaynak* ve *kesilemezlik* kanallarından da kaynaklanır. Bu projede şu girişim mekanizmaları hipotez olarak ele alınacak ve ölçümlerle ayrıştırılmaya çalışılacaktır:

- **Önbellek (cache) kirlenmesi:** AI görevinin komut/veri önbelleğini doldurarak kontrol görevinin önbellek ıskası (cache miss) yaşamasına ve yürütme süresinin artmasına yol açması. Bu etkinin gözlemlenebilmesi için deneyler I/D-cache içeren bir Cortex-M (ör. Cortex-M7/H7 sınıfı) üzerinde yürütülecektir.
- **Veri yolu / bellek çekişmesi:** Flash bekleme durumları, DMA kanalı paylaşımı ve paylaşılan periferik kilitleri nedeniyle oluşan gecikmeler [2].
- **Kritik bölümler ve kesme gecikmesi:** Kütüphane kodundaki kritik bölümlerin veya uzun kesme servis rutinlerinin kontrol görevinin başlama anını geciktirmesi [7].

4.5 Ölçüm Yöntemi

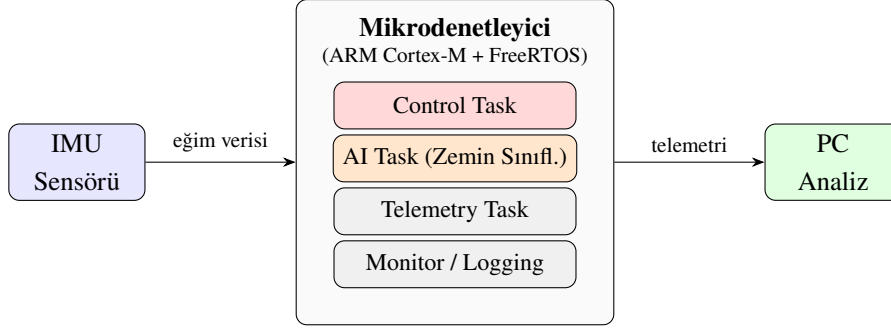
Kontrol görevinin başlangıç ve bitiş zamanları, mikrodenetleyici üzerindeki çevrim sayacı (DWT CYCCNT) veya yüksek çözünürlüklü bir zamanlayıcı kullanılarak ölçülecektir. Her kontrol döngüsü için şu değerler kaydedilecektir: görev başlangıç zamanı, görev bitiş zamanı, görev yürütme süresi, beklenen periyottan sapma, deadline kaçırma durumu, robot eğim açısı ve AI çıkarım süresi.

Ölçümün geçerliliği için iki ilke gözetilecektir. Birincisi *karıştırıcı değişken kontrolü*: denge kalitesi yalnızca zamanlamaya değil; PID ayarına, batarya voltaj düşüşüne ve mekanik gevşekliğe de bağlıdır. Bu nedenle tüm senaryolar aynı robot, aynı PID kazançları ve denetlenen batarya seviyesi altında tekrarlanacaktır. İkincisi *ölçüm müdahalesinin sınırlandırılması*: telemetri trafiği jitter kaynağı olabileceğinden, veriler tamponlanıp en düşük öncelikli görevle boşaltılacak ve ölçüm altyapısının kendi etkisi ayrıca değerlendirilecektir.

Toplanan veriler UART/USB üzerinden PC'ye aktarılacak ve Python tabanlı analiz scriptleri ile işlenecektir. Jitter bir dağılım olarak ele alınacak; ortalama ve en kötü durum değerlerinin yanı sıra p99 / p99.9 kuyruk gecikmeleri, histogramlar ve tekrar sayısına dayalı tekrarlanabilirlik raporlanacaktır.

5 Mimari Taslak

Sistemin yüksek seviyeli mimarisi Şekil 1'de verilmiştir. IMU sensörü robotun eğim bilgisini mikrodenetleyiciye iletir. Mikrodenetleyici üzerinde FreeRTOS çalışır ve görevler önceliklendirilmiş şekilde yürütülür. Control Task kritik görevdir ve robotun dengesinden sorumludur. AI Task, zemin sınıflandırması yapan TinyML çıkarım iş yükünü temsil eder. Telemetry Task ölçüm verilerini PC tarafına aktarır. PC tarafında veriler analiz edilerek jitter, deadline miss ve denge kalitesi grafikleri üretilir.



Şekil 1: Sistemin yüksek seviyeli mimarisi.

6 Deney Senaryoları

Proje kapsamında Tablo 3’deki deney senaryoları uygulanacaktır. Bu senaryolar sayesinde sistemin yalnızca AI açık/kapalı durumları değil; farklı önceliklendirme, zamanlama stratejisi ve izolasyon yaklaşımları altındaki davranışı da karşılaştırılacaktır.

Tablo 3: Deney senaryoları

Senaryo	Açıklama	Amaç
S0	AI görevi yok; yalnızca kontrol ve telemetri görevleri çalışıyor	Temel jitter ve deadline değerlerini belirlemek
S1	AI görevi düşük öncelikli çalışıyor	AI yükünün kontrol görevini paylaşılan kaynak üzerinden ne ölçüde etkilediğini test etmek
S2	AI görevi kontrol göreviyle aynı öncelikte çalışıyor	Yanlış önceliklendirme durumunda jitter artışını gözlemlemek
S3	AI görevi kontrol görevinden daha yüksek öncelikte çalışıyor	Kritik kontrol görevinin engellenmesi durumunda deadline miss etkisini ölçmek
S4	AI görevi zaman pencereli veya parçalara bölünmüş şekilde çalışıyor	Zamanlama stratejilerinin deterministik davranışı iyileştirip iyileştirmediğini incelemek
S5	AI görevi MPU izolasyonu altında çalışıyor	Bellek izolasyonunun hata yayılımını ve girişimi sınırlandırma etkisini değerlendirmek

Her senaryo, üç AI yük seviyesi (hafif/orta/ağır) için tekrarlanacak ve istatistiksel geçerlilik amacıyla yeterli sayıda koşu alınacaktır.

7 Kapsam ve Başarı Kriterleri

7.1 Kapsam Dahili Konular

- Önbellekli ARM Cortex-M tabanlı mikrodenetleyici üzerinde FreeRTOS kullanımı
- İki tekerlekli denge robotu prototipi geliştirilmesi
- IMU tabanlı açı ölçümü ve filtreleme

- PID tabanlı denge kontrolü
- Zemin sınıflandırması TinyML görevinin sisteme entegrasyonu (üç yük seviyesi)
- Kontrol döngüsü jitter ve deadline ölçümü
- AI iş yükünün farklı önceliklerde ve zamanlama stratejilerinde test edilmesi
- PC tarafında veri analizi ve grafik üretimi
- Temel MPU izolasyonu veya izolasyon etkisinin deneysel gösterimi

7.2 Kapsam Dışı Konular

- Endüstriyel seviyede güvenlik sertifikasyonu
- Tam formal WCET (en kötü durum yürütme süresi) analizi
- Üretim seviyesinde secure boot altyapısı
- Karmaşık görüntü işleme tabanlı yapay zekâ modelleri
- Zemin tahminine göre kontrolün uyarlanması (kapalı-çevrim AI kullanımı)
- Gelişmiş otonom navigasyon
- Ticari ürün kalitesinde mekanik tasarım
- Bulut bağlantılı IoT platformu geliştirme

7.3 Ölçülebilir Başarı Kriterleri

1. Robotun AI görevi kapalıyken en az 30 saniye boyunca dengede kalabilmesi.
2. Control Task'ın FreeRTOS üzerinde 1–2 ms aralığında periyodik çalıştırılabilmesi.
3. AI açık ve kapalı senaryolarda kontrol döngüsü jitter değerlerinin – ortalama, en kötü durum ve p99/p99.9 kuyruk gecikmeleri dahil – ölçülüp karşılaştırılabilmesi.
4. En az üç farklı AI zamanlama senaryosu için deadline miss oranının, koşu sayısı ve değişkenlikle birlikte raporlanması.
5. AI (zemin sınıflandırması) çıkarım süresinin her senaryoda ölçülmesi, ortalama/en kötü durum değerlerinin çıkarılması ve sınıflandırma doğruluğunun ikincil bir metrik olarak raporlanması.
6. Robot eğim açısı RMS hatasının AI yükü açık ve kapalı durumlarda, sabit PID kazançları altında karşılaştırılması.
7. Toplanan verilerden en az beş farklı, açıklayıcı grafik veya tablo üretilmesi.
8. Proje sonunda çalışan fiziksel demo, kaynak kod deposu, teknik rapor ve sunum teslim edilmesi.

8 Çalışma Planı ve İş Bölümü

8.1 Çalışma Planı

Proje dokuz aylık bir süre için planlanmıştır. Çalışma planı Tablo 4’de verilmiştir. Risk yönetimindeki temel yaklaşım, önce minimum çalışan sistemi oluşturmak; ardından AI ve izolasyon katmanlarını kademeli eklemektir.

Tablo 4: Çalışma planı

Ay	Çalışma	Beklenen Çıktı
1	Literatür araştırması, donanım seçimi (önbellekli MCU), C ve mikrodenetleyici temelleri	Ön araştırma, donanım listesi, geliştirme ortamı
2	GPIO, UART, timer, PWM, interrupt ve FreeRTOS temel görevleri	Temel FreeRTOS demo sistemi
3	IMU entegrasyonu, açı hesaplama, PID algoritması	Sensör okuma ve kontrol modülü
4	Robot mekanik/elektronik entegrasyonu, AI olmadan denge kontrolü	Temel çalışan denge robotu
5	Zamanlama ölçüm altyapısı, jitter ve deadline ölçümleri	Telemetri ve CSV veri toplama sistemi
6	Zemin verisi toplama, model eğitimi ve TinyML çıkarım görevinin (yük seviyeleriyle) entegrasyonu	Zemin sınıflandırma modeli, AI Task ve çıkarım süresi ölçümü
7	Farklı öncelik ve zamanlama senaryolarının deneyleri	Karşılaştırmalı deney verileri
8	MPU izolasyonu, hata senaryoları ve ek deneyler	İzolasyon değerlendirmesi ve risk senaryoları
9	Veri analizi, rapor, sunum ve demo hazırlığı	Final rapor, demo videosu, sunum

8.2 İş Bölümü

Takım, iki ana teknik hat etrafında örgütlenmiştir: *donanım* ve *yapay zekâ*. Donanım hattı robotun fiziksel ve gömülü altyapısından; yapay zekâ hattı ise TinyML zemin sınıflandırma görevinden sorumludur. Üyelerin ana sorumluluk alanları Tablo 5’de özetlenmiştir. Bu iki hattın dışında kalan ortak görevler (Tablo 6), gerçek zamanlı kontrol döngüsünün ve ölçüm altyapısının projenin çekirdeğini oluşturması nedeniyle dört üye arasında eşit olarak paylaşılır ve birlikte yürütülür.

Tablo 5: Takım üyeleri ve ana sorumluluk alanları

Takım Üyesi	Ana Hat	Başlıca Sorumluluklar
Ali Hakan Kıncal	Donanım	MCU/donanım seçimi, robot mekanik-elektronik entegrasyonu, güç hattı tasarımı ve erken testi
Barış Emre Ahi	Donanım	IMU donanım entegrasyonu, motor sürücü ve PWM sürme, şasi montajı ve donanım doğrulama
Uğur Talan	Yapay Zekâ	Zemin verisi toplama ve etiketleme, model eğitimi (PC), model boyutu ve yük seviyelerinin tasarımı
Mert Can Ayhan	Yapay Zekâ	TensorFlow Lite Micro / CMSIS-NN entegrasyonu, int8 nicemleme, AI Task’ın FreeRTOS’a gömülmesi

Tablo 6: Tüm üyeler tarafından eşit paylaşılan ortak görevler

Ortak Görev	Açıklama
FreeRTOS görev yapısı ve PID kontrol	Görevlerin önceliklendirilmesi, kontrol döngüsünün gerçekleşmesi ve ayarlanması
Zamanlama ölçüm altyapısı	DWT CYCCNT ile jitter/deadline ölçümü ve UART/USB telemetri sistemi
Deney senaryolarının yürütülmesi	S0–S5 senaryolarının üç yük seviyesinde, yeterli koşu sayısı ile çalıştırılması
MPU izolasyonu deneyleri	Bellek izolasyonu kurulumu ve girişim sınırlandırma etkisinin değerlendirilmesi
Veri analizi ve raporlama	Python analiz scriptleri, grafikler, teknik rapor, sunum, demo ve proje web sitesi

Donanım ve yapay zekâ hatları paralel ilerlese de, üyeler kendi hatları dışındaki ortak görevlerde de aktif rol alır; böylece bilgi tek bir kişide toplanmaz ve herhangi bir üyenin yokluğunda proje aksamadan sürdürülebilir.

9 Riskler ve Önlemler

Proje kapsamındaki başlıca riskler ve önlemler Tablo 7’te verilmiştir.

Tablo 7: Risk analizi

Risk	Olasılık	Etki	Önlem / B Planı
Gömülü sistem deneyiminin düşük olması	Yüksek	Yüksek	İlk iki ay temel C, ARM Cortex-M ve FreeRTOS öğrenimine ayrılacaktır.
Robotun dengede kal-maması	Orta	Yüksek	Önce AI olmadan basit RTOS kontrolü geliştirilecek, hazır şasi kullanılacaktır.
IMU verisinin gürültülü olması	Orta	Orta	Complementary filter kullanılacak, gerekirse Kalman filter değerlendirilecektir.
Zemin sınıfları arasında titreşim imzasının ayırt edilememesi	Orta	Orta	Belirgin biçimde farklı yüzeyler seçilecek; doğruluk düşse de görev tekrarlanabilir bir iş yükü olarak deneyde kullanılmaya devam edecektir.
Motor veya güç sistemi-nin yetersiz kalması	Orta	Yüksek	Yedek motor sürücü ve uygun batarya seçilecek, güç hattı erken test edilecektir.
Girişim etkisinin ölçü-lemeyecek kadar küçük çıkması	Orta	Yüksek	Önbellekli MCU seçilerek cache kirlenmesi etkisi gözlemlenebilir kılınacak; ağır AI yükü ile etki büyütülecektir.
TinyML modelinin fazla ağır olması	Orta	Orta	Hafif/orta/ağır kontrollü AI yük seviyeleri oluşturulacaktır.
MPU entegrasyonunun zor olması	Yüksek	Orta	MPU ana başarı kriteri değil, genişletilmiş hedef olarak ele alınacaktır.
Veri toplama altyapısı-nın geç tamamlanması	Orta	Yüksek	Zamanlama ölçümü ve telemetri beşinci ayda tamamlanacaktır.
Son ay yeni özellik ek-lenmesi	Orta	Yüksek	Dokuzuncu ay yalnızca test, analiz, raporlama ve sunuma ayrılacaktır.

Risk yönetiminde temel yaklaşım, önce minimum çalışan sistemi oluşturmak, ardından AI ve güvenlik katmanlarını kademeli olarak eklemektir. Böylece proje, tek bir zorlu bileşene bağlı kalmadan ilerleyebilecektir.

10 Beklenen Çıktılar

Proje sonunda aşağıdaki somut çıktılar elde edilecektir:

- Önbellekli ARM Cortex-M tabanlı çalışan denge robotu prototipi
- FreeRTOS tabanlı çok görevli gömülü yazılım
- IMU ve PID tabanlı denge kontrol modülü
- Zemin sınıflandırması TinyML görevi (üç yük seviyesi)
- Jitter, yürütme süresi ve deadline ölçüm altyapısı
- UART/USB tabanlı telemetri sistemi
- Python tabanlı veri analiz scriptleri
- AI açık/kapalı ve farklı öncelik senaryolarına ait karşılaştırmalı grafikler (kuyruk gecikmeleri dahil)

- MPU izolasyonu deneyi veya teknik değerlendirmesi
- Açık kaynak kod deposu, final proje raporu, demo videosu ve final sunumu

11 Sonuç

Bu proje, gerçek zamanlı kontrol ile kenar yapay zekâ çıkarımının aynı kaynak-kısıtlı gömülü sistem üzerinde birlikte çalıştırılmasının zamanlama etkilerini deneysel olarak incelemeyi amaçlamaktadır. İki tekerlekli denge robotu, kontrol döngüsündeki gecikmelerin fiziksel olarak gözlemlenebilmesini sağladığı için uygun bir deney platformu sunmaktadır. Yapay zekâ görevi olarak seçilen zemin sınıflandırması, hem anlamlı bir çıkarım işi yapması hem de model boyutuyla ölçeklenebilir, tekrarlanabilir bir iş yükü sunması bakımından deneyin amacına uygundur.

Projenin temel katkısı, TinyML çıkarım görevinin FreeRTOS tabanlı gerçek zamanlı kontrol döngüsü üzerindeki jitter, deadline miss ve denge kalitesi etkilerini ölçülebilir hâle getirmesi; ve bu etkinin *hangi mekanizmalardan* (cache kirlenmesi, kaynak çekişmesi, kritik bölümler) kaynaklandığını hipotezleştirip ayırtmaya çalışmasıdır. Ayrıca görev önceliklendirme, zaman pencereli çalışma ve MPU tabanlı izolasyon gibi yöntemlerle bu etkinin nasıl sınırlandırılabilceği karşılaştırmalı olarak değerlendirilecektir.

Başarıyla tamamlandığında proje, yalnızca çalışan bir robot prototipi değil; aynı zamanda gerçek zamanlılık, determinizm, görev önceliklendirme, kenar yapay zekâ ve emniyet-kritik gömülü sistemler konusunda karıştırıcı değişkenleri denetlenmiş, istatistiksel olarak raporlanmış deneysel veri üreten kapsamlı bir mühendislik çalışması ortaya koyacaktır.

Kaynaklar

- [1] Z. Chang, S. Liu, X. Xiong, Z. Cai, and G. Tu, “A survey of recent advances in edge-computing-powered artificial intelligence of things,” *IEEE Internet of Things Journal*, vol. 8, no. 18, pp. 13849–13875, 2021.
- [2] R. Tabish, R. Pellizzoni, R. Mancuso, G. Gracioli, R. Miroslou, and M. Caccamo, “X-Stream: Accelerating streaming segments on MPSoCs for real-time applications,” *Journal of Systems Architecture*, vol. 138, art. 102857, 2023.
- [3] E. Li, L. Zeng, Z. Zhou, and X. Chen, “Edge AI: On-demand accelerating deep neural network inference via edge computing,” *IEEE Transactions on Wireless Communications*, vol. 19, no. 1, pp. 447–457, 2019.
- [4] P. Jain, N. Pateria, G. Anjum, A. Tiwari, and A. Tiwari, “Edge AI and on-device machine learning for real time processing,” *International Journal of Innovative Research in Computer and Communication Engineering*, vol. 12, pp. 8137–8146, 2024.
- [5] R. David, M. Duke, S. Jain, V. J. Reddi, N. Jeffries, J. Li, *et al.*, “TensorFlow Lite Micro: Embedded machine learning for TinyML systems,” *Proceedings of Machine Learning and Systems*, vol. 3, pp. 800–811, 2021.

- [6] S. Deng, H. Zhao, W. Fang, J. Yin, S. Dustdar, and A. Y. Zomaya, “Edge intelligence: The confluence of edge computing and artificial intelligence,” *IEEE Internet of Things Journal*, vol. 7, no. 8, pp. 7457–7469, 2020.
- [7] O. Horst, J. Wiesböck, R. Wild, and U. Baumgarten, “Quantifying the latency and possible throughput of external interrupts on cyber-physical systems,” in *Proc. Workshop on Benchmarking Cyber-Physical Systems and Internet of Things (CPS-IoTBench)*, 2020. arXiv:2009.00506.
- [8] S. Lin, Z. Zhou, Z. Zhang, X. Chen, and J. Zhang, *Edge Intelligence in the Making: Optimization, Deep Learning, and Applications*. San Rafael, CA, USA: Morgan & Claypool, 2020.
- [9] Y. Wang, K. Yang, W. Wan, and H. Mei, “Adaptive energy saving algorithms for Internet of Things devices integrating end and edge strategies,” *Transactions on Emerging Telecommunications Technologies*, vol. 32, no. 8, art. e4122, 2021.
- [10] V. Devadas and H. Aydin, “On the interplay of voltage/frequency scaling and device power management for frame-based real-time embedded applications,” *IEEE Transactions on Computers*, vol. 61, no. 1, pp. 31–44, 2012.
- [11] N. Chong and B. Jacobs, “Formally verifying FreeRTOS’ interprocess communication mechanism,” in *Proc. Embedded World Conference*, Nürnberg, Germany, 2021.
- [12] K. K. Boddula, K. Mahapatra, A. K. Swain, and J. P. Mohanty, “Advanced IoT-based pollution monitoring: Harnessing Zephyr RTOS and AWS for real-time data management,” in *2024 IEEE 21st India Council International Conference (INDICON)*. IEEE, 2024, pp. 1–6.
- [13] J. Liu, X. Gao, B. Jiang, S. Yang, and Z. Zhang, “Deterministic replay for multi-core VxWorks applications,” in *2017 International Conference on Dependable Systems and Their Applications (DSA)*. IEEE, 2017, pp. 118–125.
- [14] N. Garg, D. Singh, and M. S. Goraya, “Energy and resource efficient workflow scheduling in a virtualized cloud environment,” *Cluster Computing*, vol. 24, no. 2, pp. 767–797, 2021.
- [15] A. Musaddiq, Y. B. Zikria, O. Hahm, H. Yu, A. K. Bashir, and S. W. Kim, “A survey on resource management in IoT operating systems,” *IEEE Access*, vol. 6, pp. 8459–8482, 2018.
- [16] Amazon Web Services, “FreeRTOS Memory Protection Unit (MPU) support,” *FreeRTOS Documentation*. [Çevrimiçi]. Erişim: <https://www.freertos.org/Security/>
- [17] P. Warden and D. Situnayake, *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. Sebastopol, CA, USA: O’Reilly Media, 2019.
- [18] C. Banbury *et al.*, “MLPerf Tiny benchmark,” in *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021.
- [19] K. Laddach, R. Łangowski, and T. Zubowicz, “Estimation of the angular position of a two-wheeled balancing robot using a real IMU with selected filters,” *Bulletin of the Polish Academy of Sciences: Technical Sciences*, vol. 70, 2022. arXiv:2309.12169.

- [20] K. J. Åström and R. M. Murray, *Feedback Systems: An Introduction for Scientists and Engineers*. Princeton, NJ, USA: Princeton University Press, 2008.
- [21] G. C. Buttazzo, *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*, 3rd ed. New York, NY, USA: Springer, 2011.